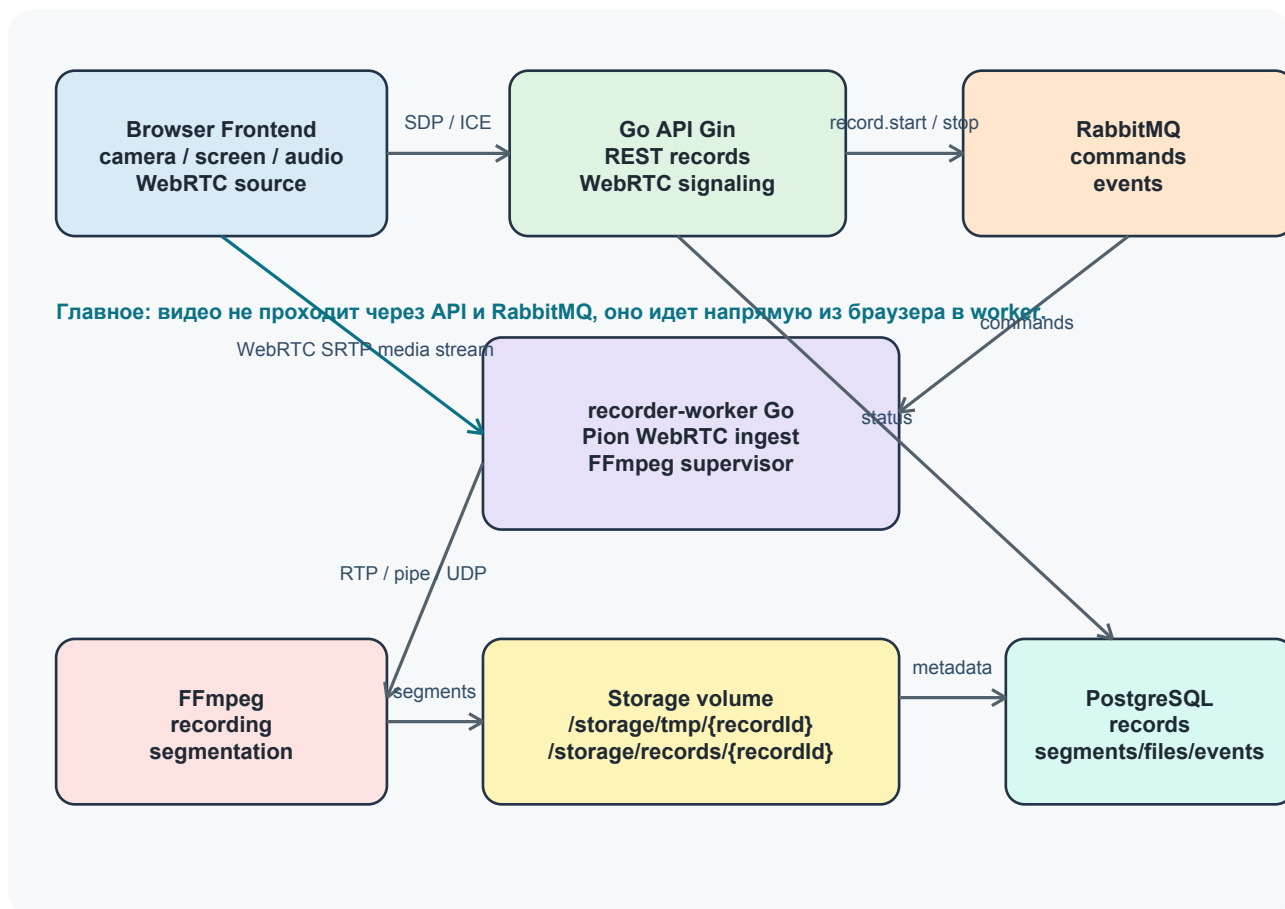


recorder-worker-scheme

Окончательная схема записи видеопотока из браузера через WebRTC с Go recorder-worker, FFmpeg, RabbitMQ, PostgreSQL и локальным storage volume.



1. Итоговое решение

Источник видеопотока — браузер пользователя. API не принимает сам видеопоток и не прокачивает медиа через RabbitMQ. API управляет записью, создает записи в PostgreSQL, выполняет WebRTC signaling и отправляет команды worker-у через RabbitMQ.

Основной media-path фиксируем так:

Browser → WebRTC → recorder-worker → FFmpeg → storage volume

Управляющий контур фиксируем отдельно:

Frontend → Go API Gin → PostgreSQL / RabbitMQ → recorder-worker

2. Роли компонентов

Компонент	Ответственность
Browser Frontend	Получает camera/screen/audio через getUserMedia или getDisplayMedia, создает RTCPeerConnection, отправляет SDP offer и ICE candidates, передает WebRTC stream.
Go API Gin	REST API для start/end/list/read, создание record, валидация, signaling endpoints, публикация команд в RabbitMQ и чтение состояния записи.
RabbitMQ	Только команды и события: start, stop, abort, started, segment.created, finished, failed. Видео через очередь не передается.
recorder-worker Go	Отдельный Go-сервис: принимает команды, поднимает WebRTC ingest через Pion, управляет процессом FFmpeg, пишет статусы и события.
FFmpeg	Получает RTP/pipe/local UDP от worker-а, выполняет запись, сегментацию, финализацию файлов и manifest/playlist.
Storage volume	Хранит временные и финальные файлы записи. Монтируется в worker на запись, в API может монтироваться read-only для отдачи файлов или ссылок.
PostgreSQL	Хранит record, record_segment, record_file, record_event: статусы, сегменты, файлы, ошибки и события.

3. Основной сценарий записи

- Frontend вызывает POST /api/records/start.
- API создает record со статусом starting, валидирует входные данные и публикует record.start в RabbitMQ.
- API возвращает recordId и URL для WebRTC signaling.
- Браузер получает media stream, создает RTCPeerConnection, добавляет audio/video tracks и отправляет SDP offer.
- API передает offer в recorder-worker, worker создает Pion PeerConnection и возвращает SDP answer.
- Браузер начинает передавать WebRTC SRTP stream напрямую в recorder-worker.
- Worker запускает FFmpeg и передает ему поток как RTP, pipe или local UDP.
- FFmpeg пишет сегменты во временную директорию storage volume.
- Готовые сегменты переносятся в финальную директорию, а worker пишет record_segment и record_file.
- При POST /api/records/end API публикует record.stop, worker останавливает WebRTC/FFmpeg, финализирует файлы и переводит record в finished или failed.

4. API endpoints

POST /api/records/start

Создает запись и возвращает recordId + данные WebRTC signaling.

POST /api/records/{recordId}/webrtc/offer

Принимает SDP offer от браузера и возвращает SDP answer от recorder-worker.

POST /api/records/end

Останавливает запись. Тело запроса:

```
{ "recordId": "uuid", "reason": "client_stop" }
```

GET /api/records

Возвращает список записей.

GET /api/records/{id}

Возвращает карточку записи, файлы, сегменты и события.

Для MVP достаточно REST signaling без trickle ICE. Для более стабильной работы за NAT и в сложных сетях лучше добавить WebSocket signaling с передачей ICE candidates в реальном времени.

5. RabbitMQ

RabbitMQ используется для надежного управляющего контура. Через него не нужно передавать видеоданные: это создаст лишнюю задержку, нагрузку и риск потери производительности.

exchange: recorder.commands
queue: recorder.worker.commands
routing keys:
record.start
record.stop
record.abort

exchange: recorder.events
routing keys:
record.started
record.segment.created
record.finished
record.failed

dead-letter:
exchange: recorder.dlx
queue: recorder.dead

worker settings:
durable queues
persistent messages
manual ack
prefetch=1 для MVP

6. Storage volume

Storage volume должен быть отдельной частью Docker-инфраструктуры. Worker монтирует его на запись, API при необходимости может монтировать его только для чтения, чтобы отдавать файлы или ссылки.

./dockers/storage/data:/storage

Временные файлы записи:
/storage/tmp/{recordId}/

Финальные файлы записи:
/storage/records/{recordId}/segments/
/storage/records/{recordId}/manifest.json
/storage/records/{recordId}/playlist.m3u8
/storage/records/{recordId}/final.mp4 # если нужна финальная склейка

Сегмент лучше считать готовым только после закрытия файла FFmpeg. После этого worker атомарно переносит его из tmp в records и добавляет строку в record_segment.

7. PostgreSQL модель состояния

Существующие таблицы остаются центральным источником правды по записи. Их можно использовать так:

Таблица	Назначение
record	recordId, conferenceId, status, reason, startedAt, stoppedAt, errorMessage, параметры качества.
record_segment	Каждый готовый сегмент: номер, путь, duration, size, checksum, статус.

Таблица	Назначение
record_file	Итоговые файлы: manifest.json, playlist.m3u8, final.mp4 или архив.
record_event	Аудит событий: started, ingest.connected, segment.created, stop.requested, finished, failed.

8. Статусы записи

starting -> API создал запись и отправил команду worker-y
 waiting -> worker готов принять WebRTC offer
 recording -> WebRTC соединение установлено, FFmpeg пишет поток
 stopping -> получен record.stop, идет мягкая остановка
 finished -> запись успешно завершена, файлы финализированы
 failed -> ошибка WebRTC, FFmpeg, storage или инфраструктуры

9. Структура проекта для реализации

cmd/worker/main.go
entrypoint recorder-worker

internal/infrastructure/webrtc/
Pion WebRTC ingest, PeerConnection, SDP answer, ICE handling

internal/infrastructure/ffmpeg/
запуск FFmpeg, контроль процесса, graceful stop, parsing ошибок

internal/infrastructure/storage/local/
tmp/final директории, атомарный перенос сегментов, manifest

internal/transport/rabbitmq/
consumer команд record.start/record.stop и publisher событий

internal/transport/http/
REST endpoints records и WebRTC signaling

dockers/storage/data/
локальный volume для записей

10. Ключевые правила реализации

- Видео не передавать через REST API и RabbitMQ.
- API хранит состояние и управляет жизненным циклом записи.
- Worker отвечает за media-path и должен переживать ошибки FFmpeg с понятным статусом failed.
- Каждый сегмент фиксировать в PostgreSQL только после физической готовности файла.
- Для MVP начать с одного worker-а и prefetch=1, затем добавить масштабирование и лимит одновременных записей.
- REST signaling подходит для первого прототипа; WebSocket signaling лучше для production-сценариев.

Финальный вывод

Выбранная архитектура разделяет управление и медиа. Браузер напрямую отдает WebRTC-поток в recorder-worker, worker через FFmpeg пишет сегменты в storage volume, а API, RabbitMQ и PostgreSQL обеспечивают управление, надежность, статусы и историю записи.